

EXPERIMENT-4

SACHIN SINGH

24UADS1045

Objective :-

1. WAP to evaluate the performance of implemented three-layer neural network with variations in activation functions, size of hidden layer, learning rate, batch size and number of epochs.

Description Of the Model :-

In this experiment, the performance of the three-layer neural network is evaluated by changing different hyperparameters and observing their effect on loss, accuracy, and weight updates.

Instead of keeping all parameters fixed, one hyperparameter is changed at a time while keeping others constant. This helps in understanding how each parameter influences the learning process.

The following hyperparameters were varied:

1. Activation Function
2. Hidden Layer Size
3. Learning Rate
4. Batch Size
5. Number of Epochs

1. Variation in Activation Function:-

Change the activation function in the hidden layer to compare how ReLU, Sigmoid, and Tanh affect training.

Observation:

ReLU generally gives faster convergence .

Sigmoid may suffer from vanishing gradient .

Tanh performs better than sigmoid but slower than ReLU.

2. Variation in Hidden Layer Size:-

Change number of neurons in hidden layer to observe underfitting and overfitting.

Observation:

64 neurons → lower accuracy (underfitting)

128 neurons → balanced performance

256 neurons → better accuracy but more computation

3.Variation in Learning Rate:-

Change learning rate to analyze convergence speed and stability.

Observation:

0.001 → very slow learning

0.01 → stable and good convergence

0.1 → may cause unstable loss or divergence

4. Variation in Batch Size:-

Change mini-batch size to observe smoothness of training.

Observation:

32 → noisy but faster updates

128 → balanced performance

512 → smoother curve but slower updates

5.Variation in Number of Epochs:-

Change number of epochs to observe improvement and possible overfitting.

Observation:

More epochs → lower loss

Too many epochs → risk of overfitting

Source Code :-

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt


tf.random.set_seed(42)


# Load MNIST
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()


X_train = X_train.reshape(-1, 784) / 255.0
y_train = tf.one_hot(y_train, 10)


X_train = tf.convert_to_tensor(X_train, dtype=tf.float32)
y_train = tf.convert_to_tensor(y_train, dtype=tf.float32)


def train_model(hidden_size=128,
                 activation='relu',
                 learning_rate=0.1,
                 batch_size=128,
                 epochs=5):


    # Initialize weights
    W1 = tf.Variable(tf.random.normal([784, hidden_size], stddev=0.1))
    b1 = tf.Variable(tf.zeros([hidden_size]))
    W2 = tf.Variable(tf.random.normal([hidden_size, 10], stddev=0.1))
    b2 = tf.Variable(tf.zeros([10]))
```

```
dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))
```

```
dataset = dataset.shuffle(60000).batch(batch_size)
```

```
loss_history = []
```

```
accuracy_history = []
```

```
weight_history = []
```

```
for epoch in range(epochs):
```

```
    epoch_loss = 0
```

```
    correct = 0
```

```
    total = 0
```

```
    old_W1 = W1.numpy()
```

```
    for x_batch, y_batch in dataset:
```

```
        with tf.GradientTape() as tape:
```

```
            Z1 = tf.matmul(x_batch, W1) + b1
```

```
            if activation == 'relu':
```

```
                A1 = tf.nn.relu(Z1)
```

```
            elif activation == 'sigmoid':
```

```
                A1 = tf.nn.sigmoid(Z1)
```

```
            elif activation == 'tanh':
```

```
                A1 = tf.nn.tanh(Z1)
```

```
            Z2 = tf.matmul(A1, W2) + b2
```

```
            loss = tf.reduce_mean(
```

```
                tf.nn.softmax_cross_entropy_with_logits(
```

```
        labels=y_batch,  
        logits=Z2  
    )  
)
```

```
grads = tape.gradient(loss, [W1, b1, W2, b2])
```

```
W1.assign_sub(learning_rate * grads[0])  
b1.assign_sub(learning_rate * grads[1])  
W2.assign_sub(learning_rate * grads[2])  
b2.assign_sub(learning_rate * grads[3])
```

```
epoch_loss += loss.numpy()
```

```
preds = tf.argmax(Z2, axis=1)  
labels = tf.argmax(y_batch, axis=1)
```

```
correct += tf.reduce_sum(  
    tf.cast(preds == labels, tf.float32)  
).numpy()
```

```
total += y_batch.shape[0]
```

```
acc = correct / total
```

```
loss_history.append(epoch_loss)  
accuracy_history.append(acc)
```

```
weight_change = np.linalg.norm(W1.numpy() - old_W1)
```

```
weight_history.append(weight_change)
```

```
return loss_history, accuracy_history, weight_history
```

```
relu_loss, relu_acc, _ = train_model(activation='relu')
```

```
sig_loss, sig_acc, _ = train_model(activation='sigmoid')
```

```
tanh_loss, tanh_acc, _ = train_model(activation='tanh')
```

```
plt.figure()
```

```
plt.plot(relu_loss, label="ReLU")
```

```
plt.plot(sig_loss, label="Sigmoid")
```

```
plt.plot(tanh_loss, label="Tanh")
```

```
plt.legend()
```

```
plt.title("Loss vs Activation Function")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.show()
```

```
plt.figure()
```

```
plt.plot(relu_acc, label="ReLU")
```

```
plt.plot(sig_acc, label="Sigmoid")
```

```
plt.plot(tanh_acc, label="Tanh")
```

```
plt.legend()
```

```
plt.title("Accuracy vs Activation Function")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

```
final_acc_activation = [  
    relu_acc[-1],  
    sig_acc[-1],  
    tanh_acc[-1]  
]
```

```
labels = ['ReLU', 'Sigmoid', 'Tanh']
```

```
plt.figure()  
plt.bar(labels, final_acc_activation)  
plt.title("Final Accuracy vs Activation Function")  
plt.xlabel("Activation Function")  
plt.ylabel("Final Accuracy")  
plt.show()
```

```
loss64, acc64, _ = train_model(hidden_size=64)  
loss128, acc128, _ = train_model(hidden_size=128)  
loss256, acc256, _ = train_model(hidden_size=256)
```

```
final_acc_hidden = [  
    acc64[-1],  
    acc128[-1],  
    acc256[-1]  
]
```

```
labels = ['64', '128', '256']
```

```
plt.figure()
plt.bar(labels, final_acc_hidden)
plt.title("Final Accuracy vs Hidden Layer Size")
plt.xlabel("Hidden Layer Neurons")
plt.ylabel("Final Accuracy")
plt.show()

# Train with different learning rates
loss01, acc01, _ = train_model(learning_rate=0.01)
loss1, acc1, _ = train_model(learning_rate=0.1)
loss5, acc5, _ = train_model(learning_rate=0.5)
```

```
# Plot final accuracy comparison
```

```
final_acc_lr = [
    acc01[-1],
    acc1[-1],
    acc5[-1]
]
```

```
labels = ['0.01', '0.1', '0.5']
```

```
plt.figure()
plt.bar(labels, final_acc_lr)
plt.title("Final Accuracy vs Learning Rate")
plt.xlabel("Learning Rate")
plt.ylabel("Final Accuracy")
plt.show()
```



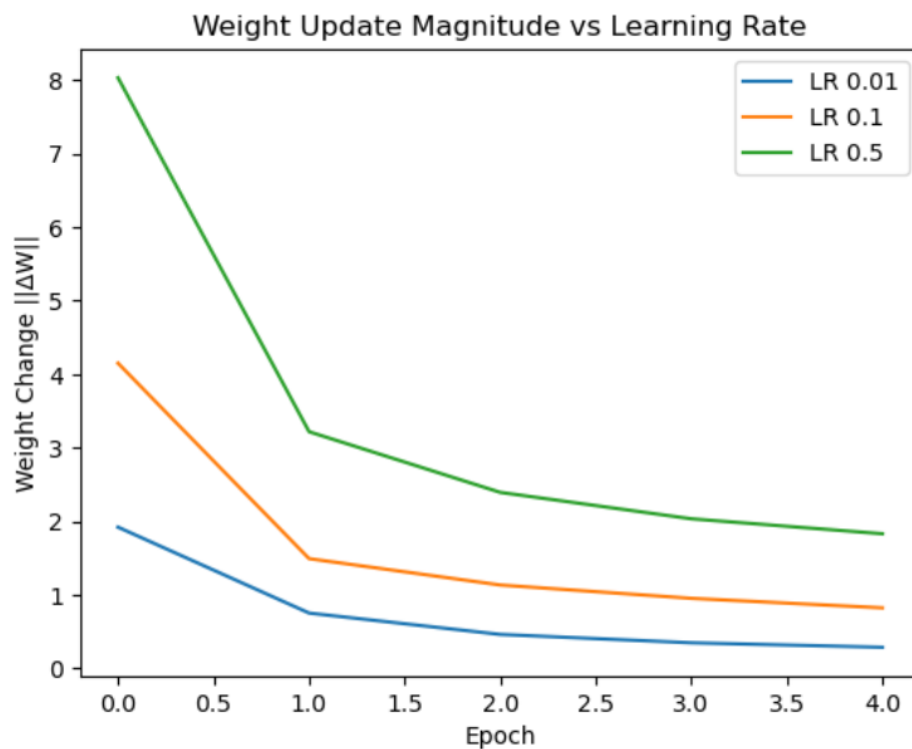
```

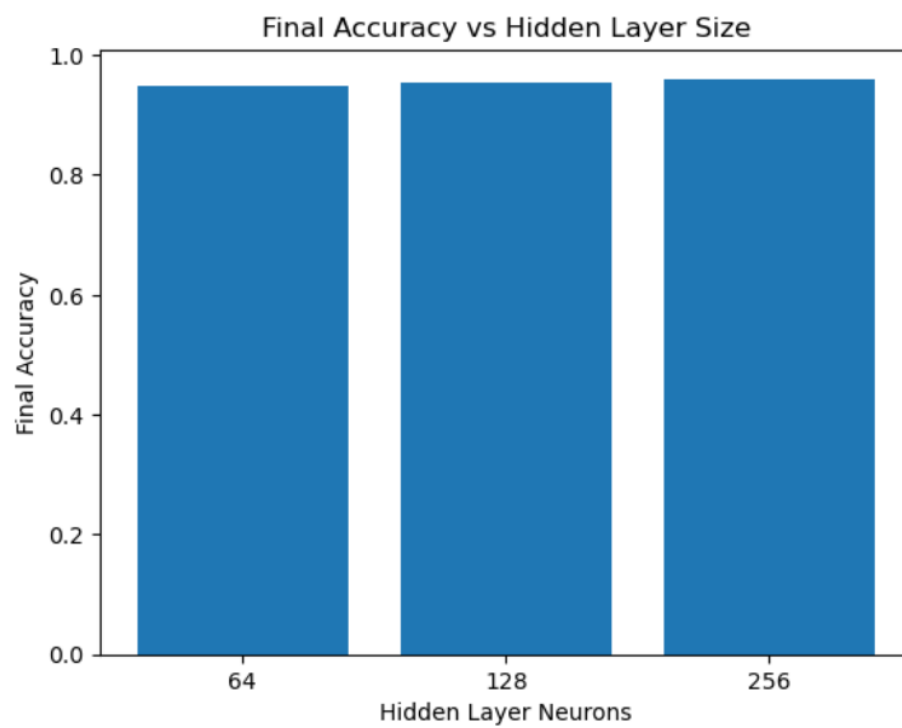
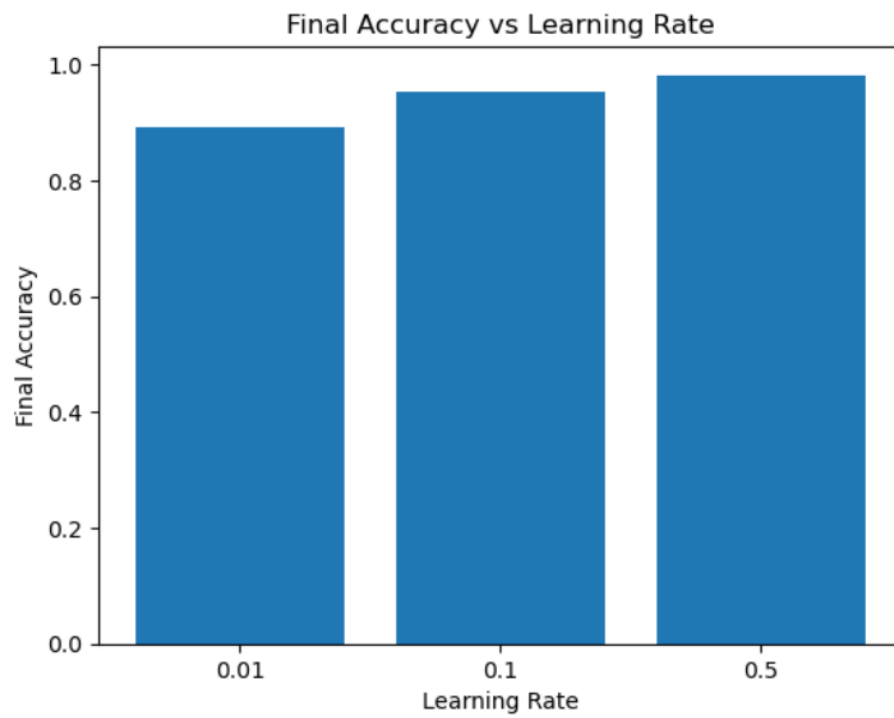
_, w01 = train_model(learning_rate=0.01)
_, w1 = train_model(learning_rate=0.1)
_, w5 = train_model(learning_rate=0.5)

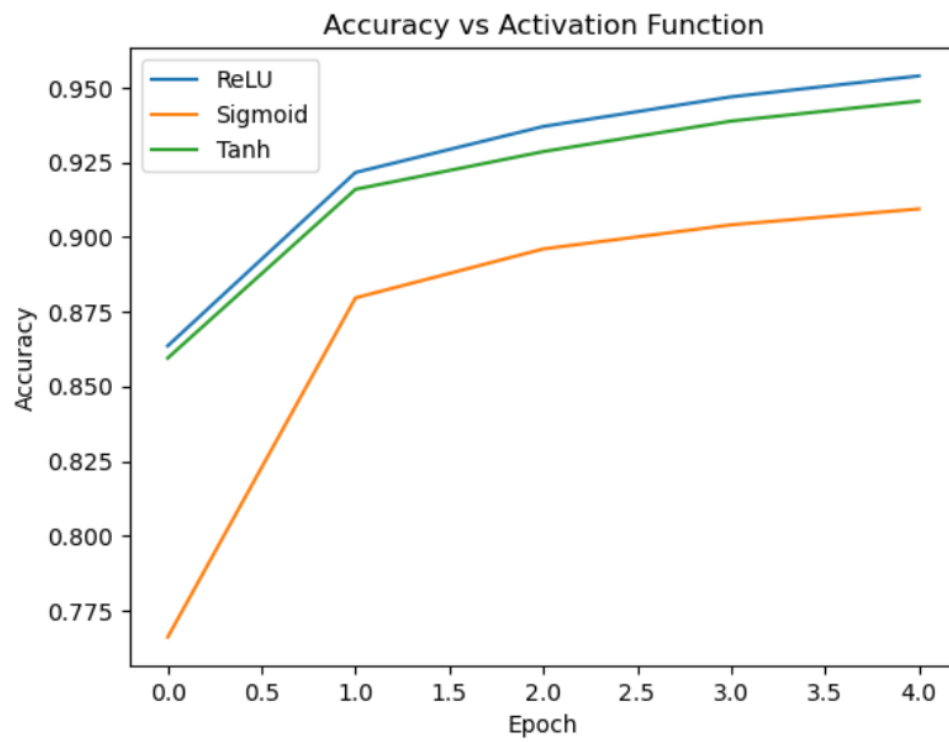
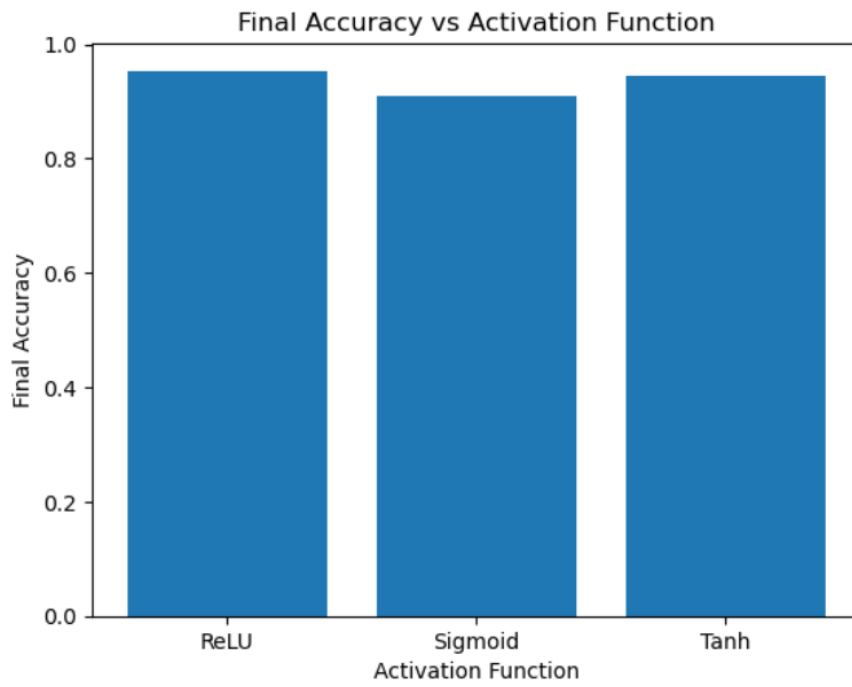
plt.figure()
plt.plot(w01, label="LR 0.01")
plt.plot(w1, label="LR 0.1")
plt.plot(w5, label="LR 0.5")
plt.legend()
plt.title("Weight Update Magnitude vs Learning Rate")
plt.xlabel("Epoch")
plt.ylabel("Weight Change  $||\Delta W||$ ")
plt.show()

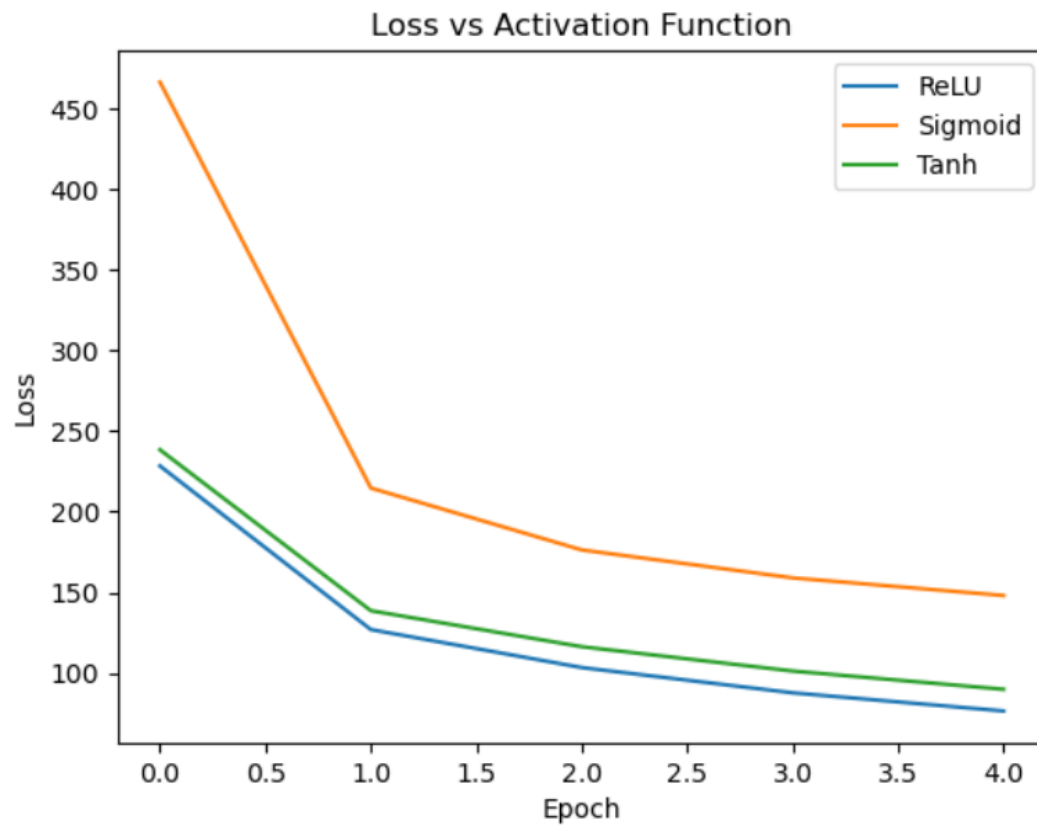
```

Output :-









Description of Code :-

In this experiment, the code is structured in a flexible and modular way so that different hyperparameters of the three-layer neural network can be changed easily and their effects can be observed.

First, the MNIST dataset is loaded and preprocessed. The images are reshaped from 28×28 format into 784 input features and normalized between 0 and 1. The output labels are converted into one-hot encoded form so that the model can perform multi-class classification.

A general training function is created to make the model reusable. Instead of fixing all parameters, the function accepts different hyperparameters as input arguments such as hidden layer size, activation function, learning rate, batch size, and number of epochs. This allows experimentation without rewriting the entire program.

Inside the training function, weights and biases for both layers are initialized randomly. The size of the hidden layer is determined based on the parameter passed. This makes it possible to test different hidden layer configurations such as 64, 128, or 256 neurons.

The forward propagation step computes the output in two stages. First, the input is multiplied with the first weight matrix and bias is added. Then the selected activation function is applied to the hidden layer output. The activation function is chosen dynamically based on the parameter passed (ReLU, Sigmoid, or Tanh). After that, the output layer computation is performed and Softmax activation is applied to produce probability values.

The model is trained using mini-batch gradient descent. In each epoch, the dataset is shuffled to avoid bias in training. The shuffled data is divided into smaller batches. For each batch, forward propagation is performed, loss is calculated using categorical cross-entropy, and gradients are computed using TensorFlow's automatic differentiation. The weights and biases are then updated using the gradient descent update rule.

After completing each epoch, the model is evaluated on the entire training dataset. The loss and accuracy values are calculated and stored. These stored values are later used to generate graphs for comparison between different hyperparameter settings.

To evaluate the influence of hyperparameters, separate experiments are conducted by changing one parameter at a time while keeping the others constant. For example, activation functions are changed while keeping learning rate and hidden size fixed. Similarly, learning rate, batch size, number of epochs, and hidden layer size are varied individually. The resulting loss and accuracy values are compared to analyze performance differences.

Overall, the code is designed in such a way that it supports systematic experimentation and helps in understanding how each hyperparameter affects convergence speed, stability, weight updates, and final accuracy of the neural network.

Performance Evaluation:-

In this experiment, the performance of the three-layer neural network was evaluated by varying different hyperparameters and observing their effect on loss and accuracy.

Initially, the model showed high loss and low accuracy, which indicates random prediction. As training progressed, the loss decreased and accuracy increased, showing that the network was learning properly.

When activation functions were compared, it was observed that ReLU gave faster convergence and better accuracy compared to Sigmoid and Tanh. Sigmoid showed slower learning due to smaller gradient values, while Tanh performed better than Sigmoid but slightly slower than ReLU.

When the hidden layer size was increased, the accuracy improved up to a certain point. A smaller hidden layer led to underfitting, while a larger hidden layer improved learning but increased computation time.

Variation in learning rate showed that a very small learning rate caused slow convergence, while a very large learning rate made the loss unstable. A moderate learning rate provided stable and efficient learning.

Changing the batch size affected the smoothness of the loss curve. Smaller batch sizes produced more fluctuations but updated weights more frequently. Larger batch sizes gave smoother curves but slower updates.

Increasing the number of epochs reduced loss further and improved accuracy. However, too many epochs may lead to overfitting if not controlled.

From the graphs of loss and accuracy, it was clear that proper tuning of hyperparameters is very important for achieving good performance. The experiment demonstrated that neural network performance depends strongly on selecting appropriate hyperparameter values.

Overall, the model achieved high accuracy and stable convergence when suitable hyperparameters were chosen.

My Comments:-

1.From this experiment, I learned that the performance of a neural network does not depend only on the architecture but also heavily on the choice of hyperparameters. Small changes in activation function, learning rate, batch size, hidden layer size, and number of epochs can significantly affect loss and accuracy.

2.By changing activation functions, I understood how ReLU helps in faster convergence compared to Sigmoid and Tanh. I observed that Sigmoid can slow down learning due to smaller gradients, while ReLU generally gives better performance for deep networks.

3.When I varied the hidden layer size, I learned that too few neurons can cause underfitting, and too many neurons increase computation and may lead to overfitting. So selecting an appropriate hidden size is important.

4.By experimenting with different learning rates, I clearly saw how a very small learning rate makes training slow, while a very large learning rate makes the loss unstable. This helped me understand the importance of choosing a proper learning rate for stable convergence.

5.Changing the batch size helped me understand the difference between noisy and smooth training behavior. Smaller batches gave more fluctuations in loss, while larger batches produced smoother curves.

6.By increasing the number of epochs, I observed that the loss decreases gradually, but too many epochs can lead to overfitting.